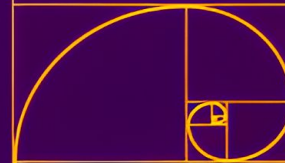


Mathematical Background

CSCI 33500: Software Design and Analysis III

Sadab Hafiz

sh3646@hunter.cuny.edu



01

But, why?

Math is a Universal
Language!



But, why?

Qualitative Comparisons

As software engineers, you might say:

- Algorithm A is *better* than Algorithm B
- Function A is *faster* than Function B

These comparisons are *qualitative*:

Of, relating to, or involving quality or kind.

- [Merriam-Webster](#)

But, why?

Problems With Qualitative Comparisons

As engineers, you won't be paid to give qualitative assessments as **qualitative statements cannot guide engineering design decisions**:

- Algorithm A could be *better* than Algorithm B, but Algorithm A would require a team of 5 engineers weeks to implement, test and integrate while Algorithm B has already been implemented and used.
- There may be circumstances where it may be beneficial to use Algorithm A, but the word *better* is not informative.

But, why?

Quantitative > Qualitative

Thus, we will look at a *quantitative* means of describing data structures and algorithms:

Of, relating to, or involving the measurement of quantity or amount.

- [Merriam-Webster](#)

This will be based on **mathematics**. Therefore, we will look at some properties which will be used again and again throughout the course.

Rounding Operations

Floor and Ceiling Functions

For any real number x :

- $\text{floor}(x) = \lfloor x \rfloor = \text{largest integer } \leq x$
 - $\lfloor 5.2 \rfloor = 5$
 - $\lfloor -8.4 \rfloor = -9$
- $\text{ceiling}(x) = \lceil x \rceil = \text{smallest integer } \geq x$
 - $\lceil 4.5 \rceil = 5$
 - $\lceil -3.9 \rceil = -3$
- The `<cmath>` header provides `std::floor()` and `std::ceil()` for floating-point types
- Reference: [Standard library header <cmath> - cppreference.com](https://en.cppreference.com/w/cpp/numeric/math/floor)

Exponents

Laws of Exponents

$$\begin{aligned}a^x \cdot a^y &= a^{x+y} \\a^x \div a^y &= a^{x-y} \\(a^x)^y &= a^{xy}\end{aligned}$$

- Derived from the basic definitions of multiplication, division and exponentiation

Logarithms

Rules of Logarithms

$$\begin{aligned}\log xy &= \log x + \log y \\ \log x/y &= \log x - \log y \\ \log x^y &= y \log x \\ \log_a x &= \frac{\log_b x}{\log_b a}\end{aligned}$$

- Derived from the rules of exponents
- Log base-e is known as $\ln$, or the natural logarithm. The constant e and $\ln$ have some special properties. More formally:
 - ◆ If $n = e^m$, we define, $m = \ln(n)$, for $n > 0$
 - ◆ Similar to C++, Many languages use $\log(x)$ for the natural logarithm and $\log_{10}(x)$ for log base-10.

Modulo

Modulo Operator (%)

→ Euclidean division is defined by:

$$a = qb + r$$

$$a \operatorname{div} b = q$$

$$a \operatorname{mod} b = r = a - qb$$

→ Note that $a \% b = a$ for $1 \leq a < b$, not 0!

→ For example, $3 \% 11 = 3 \neq 0$

Big-O Notation

Introduction

- We adopt special notation to define **upper bounds** and **lower bounds** on functions
- In Computer Science, usually the functions we are bounding are **running time**, and **memory requirements**.
- We will refer to the runtime as $T(N)$



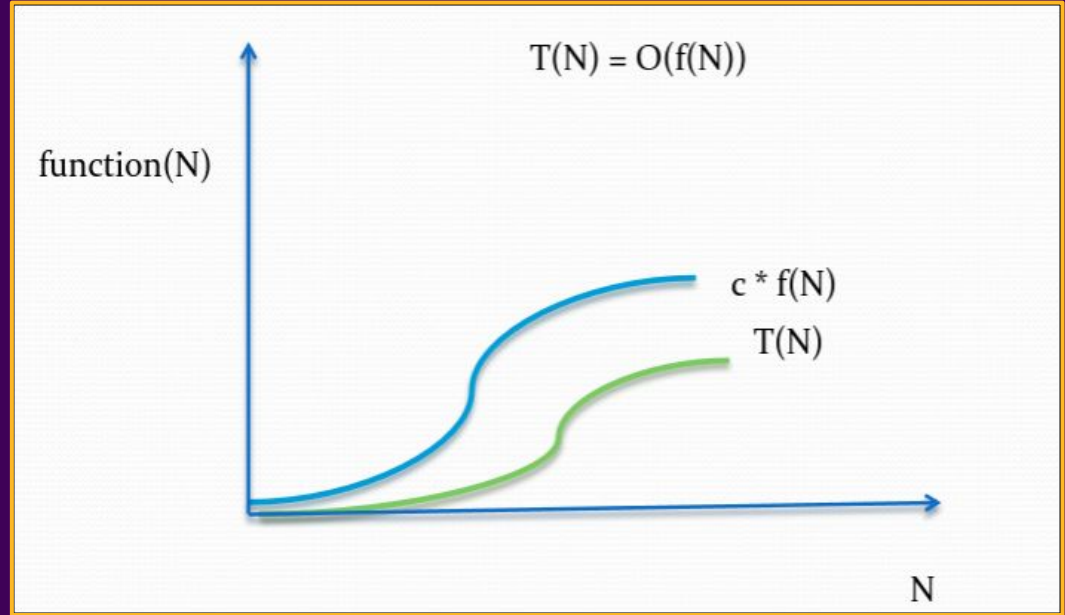
Definitions

There exist positive constants c and N_0 such that, for all $N \geq N_0$:

Notation	Bound on T(N)	Formal Definition
Big-O	Upper-Bound	$T(N) = O(f(N)) \leftarrow T(N) \leq c \cdot f(N)$
Big-Ω	Lower-Bound	$T(N) = \Omega(g(N)) \leftarrow T(N) \geq c \cdot g(N)$
Big-Θ	Tight-Bound	$T(N) = \Theta(h(N)) \leftarrow \begin{cases} T(N) = O(h(N)) \\ T(N) = \Omega(h(N)) \end{cases}$

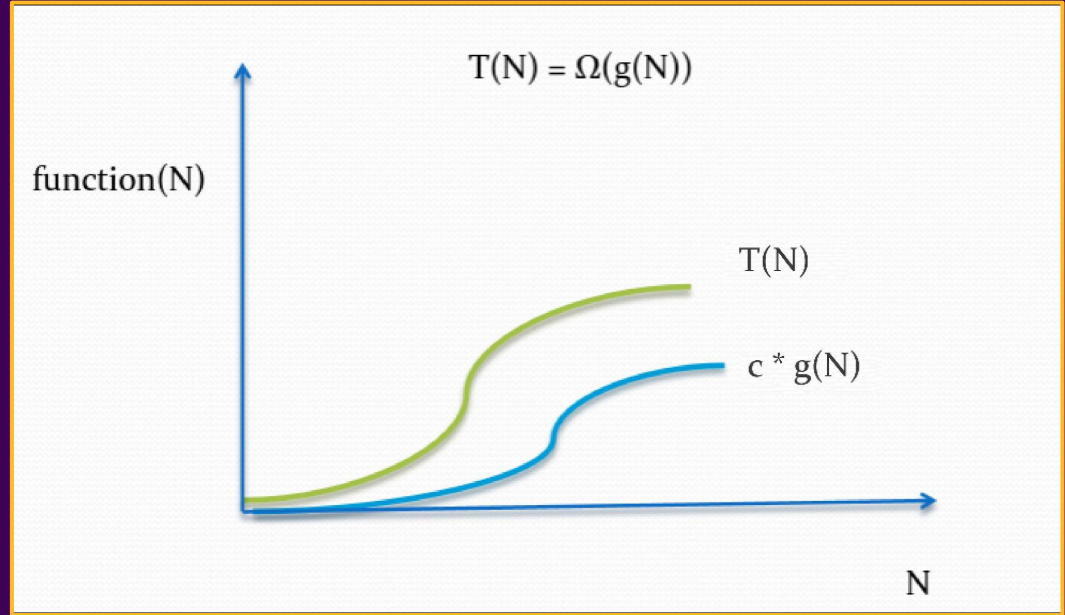
Big-O

- Example 1:
 - $N = O(N^2)$
 - Proof: $N \leq c_1 \cdot N^2$
- Example 2:
 - $4 \cdot N^2 + N = O(N^2)$
 - Proof: $4 \cdot N^2 + N \leq c_2 \cdot N^2$
- Example 3:
 - $\log(N) = O(N)$
 - Proof: $\log(N) \leq c_3 \cdot N$



Big-Ω

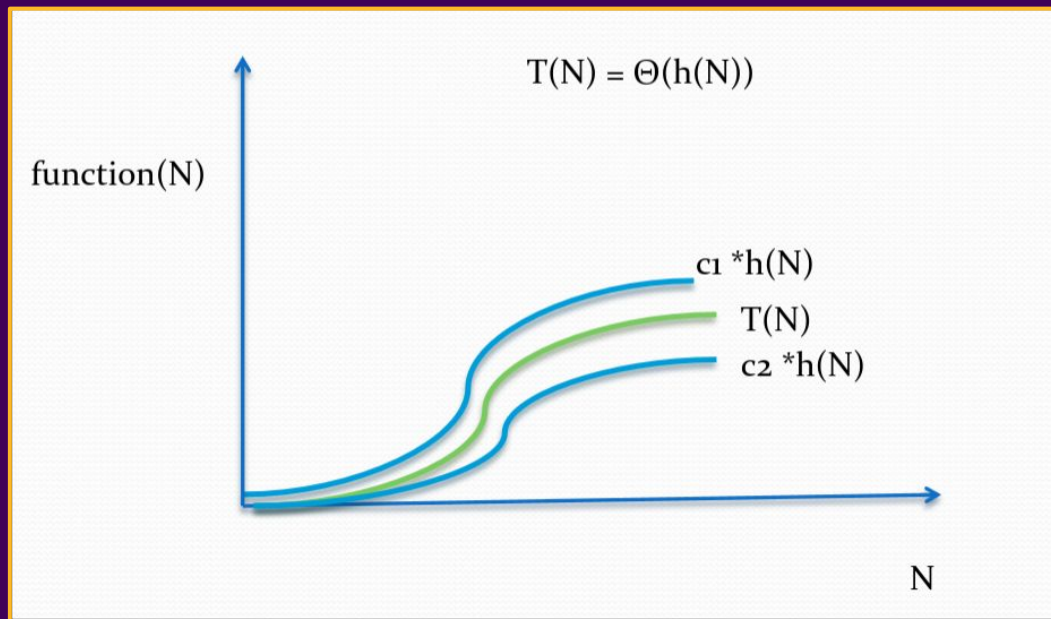
- Example 1:
 - $N^2 = \Omega(N^2)$
 - Proof: $N^2 \geq c_1 \cdot N^2$
- Example 2:
 - $N = \Omega(\log(N))$
 - Proof: $N \geq c_2 \cdot \log(N)$
- Example 3:
 - $N^3 = \Omega(N)$
 - Proof: $N^3 \geq c_3 \cdot N$



Big- Θ

- Example 1:
 - $5 \cdot N^3 + N = \Theta(N^3)$
- Example 2:
 - $3 \cdot N + 7 = \Theta(N)$

Can you prove the above?



Alternative Definition

- Alternatively, Big-O can be thought of as meaning:

$$T(N) = O(f(N)) \leftarrow \lim_{N \rightarrow \infty} f(N) \geq \lim_{N \rightarrow \infty} T(N)$$

- This is why, Big-O notation is also referred to as **asymptotic** analysis.

What is an asymptote?

One More: Little-o

- Little-o states, for **every** constant $c > 0$, there exists an N_0 such that:

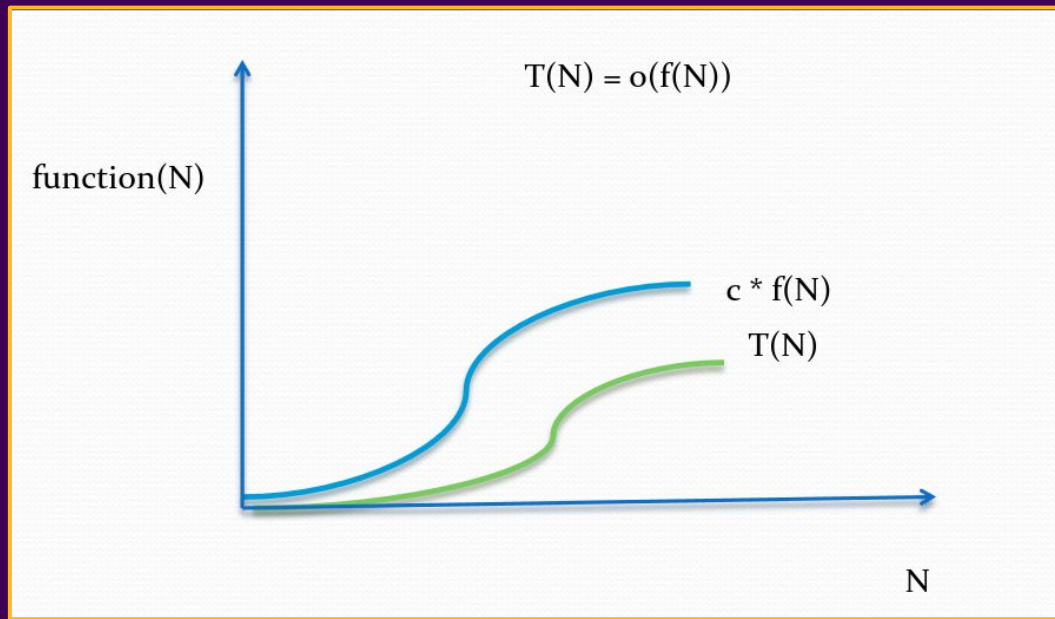
$$T(N) = o(p(N)) \leftarrow T(N) < c \cdot p(N) \quad \text{when} \quad N > N_0$$

- Little-o implies big-O because little-o is a stronger upper-bound
- Similarly, if $T(N) = o(p(N))$, $T(N) \neq \Theta(p(N))$
- Note that the growth rate of $T(N)$ for little-o is less(<) than that growth rate of $p(N)$, unlike big-O which is $\leq$ and in big- Θ which has both $\leq$ and $\geq$

Little-o

- Example 1:
 - $N = o(N^2)$
- ~~Example 2:~~
 - ~~$4 \cdot N^2 + N = o(N^2)$~~
- Example 3:
 - $\log(N) = o(N)$

How do we prove this?



Comparing Growth Rates

- Given, $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$, then:
 - Rule 1:
 - $T_1(N) + T_2(N) = O(f(N) + g(N))$
 - $T_1(N) \cdot T_2(N) = O(f(N) \cdot g(N))$
 - Rule 2:
 - If $T(N)$ is a polynomial of degree k , then $T(N) = \Theta(N^k)$
 - Rule 3:
 - $\log^k(N) = O(N)$ for any constant k
- The lower order terms do not affect the growth rate in the limit. Thus, instead of writing $O(2N^2)$ or $O(N^2 + N)$, it is just $O(N^2)$.
- If it is known that $T(N) = O(N^2)$, it is also true that $T(N) = O(N^3)$. However, it would not be considered a good estimate for the growth rate of $T(N)$.

Comparison Using Limits

In order to determine the relative growth rate of two functions $f(N)$ and $g(N)$, we can compute the following limit:

$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)}$$

If the limit is:

- **Zero:** $f(N) = o(g(N))$
- **Positive Constant:** $f(N) = \Theta(g(N))$
- **Positive Infinity (∞):** $g(N) = o(f(N))$
- **Oscillation (no limit):** limit test inconclusive

L'Hôpital's Rule

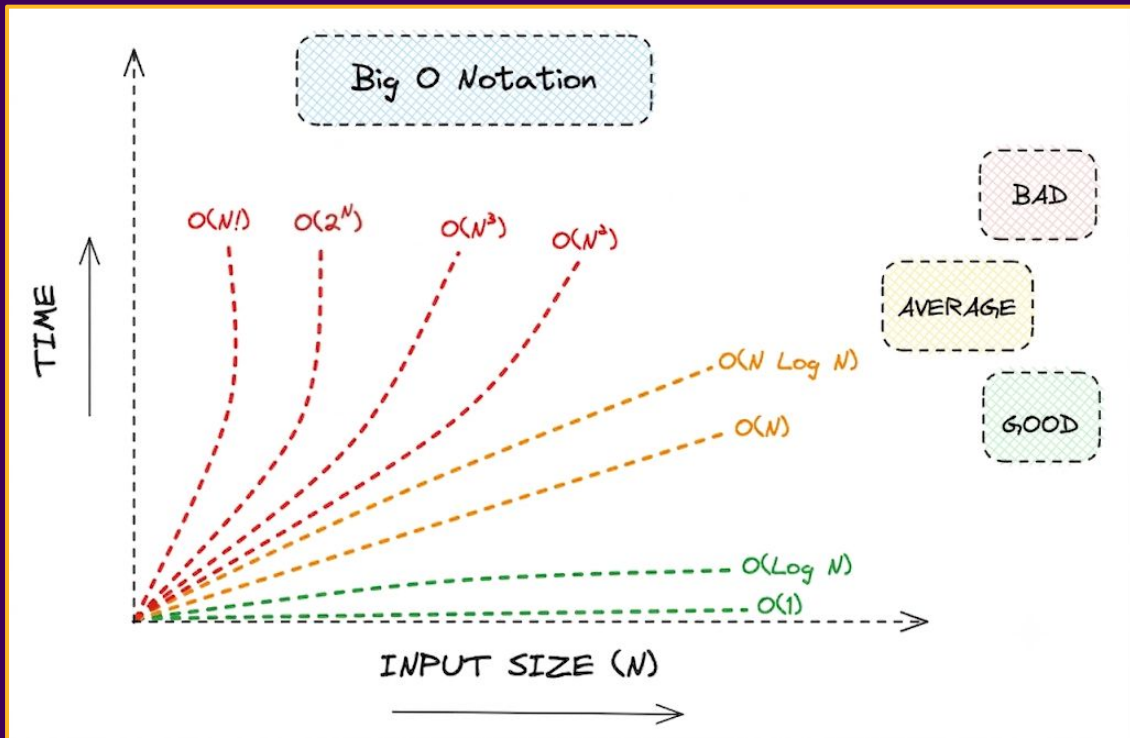
$$\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = \lim_{N \rightarrow \infty} \frac{f'(N)}{g'(N)}$$

- Sometimes, evaluating the limit leads to an indeterminate form. In such cases, we can take the derivative using L'Hôpital's Rule.
- For example: $f(N) = N^2$ vs $g(N) = N$
 - $\lim_{N \rightarrow \infty} f(N)/g(N) = \lim_{N \rightarrow \infty} N^2/N$
 - This leads to indeterminate form ∞/∞
 - Using L'Hôpital's Rule, $\lim_{N \rightarrow \infty} N^2/N = \lim_{N \rightarrow \infty} 2.N/1$
 - This limit evaluates to ∞

What is the interpretation?

Typical Growth Rates

Function	Name
c	Constant
$\log(N)$	Logarithmic
$\log^2(N)$	Log-squared
N	Linear
$N \log(N)$	Linearithmic
N^2	Quadratic
N^3	Cubic
2^N	Exponential



Model of Computation

What are we interested in?

- A model to analyze algorithms in any language (not just C++)
- One unit of time corresponds to simple instructions:
 - Addition, multiplication, comparison, assignment, division, etc.
- Fixed size (32-bit) integers
- Infinite memory
 - We are interested in the algorithm, not performance in any particular machine

What do we analyze?

Metrics

- Runtime (time complexity)
- Memory usage (space complexity)
- What kind of input?
 - Worst-case runtime
 - Best-case runtime
 - Average-case runtime
 - sometimes hard to define this
- Size of the input problem, N
 - If input is an array, N is the size
 - If input is a number, N is the number of bits required to represent it